

ISP Block Performing 3x3 Kernel Convolution Functions

Giovanni Gutierrez

Univ. of Utah ECE 6710

The architecture includes an Image Signal Processing (ISP) block that performs convolution functions per-pixel using 3x3 convolutions. The ISP block is very simplistic itself, and will run convolution at each pixel position in the image stream (once per clock cycle). The four convolution kernels are as follows.

$$\begin{array}{cc} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & -1 & 0 \\ -1 & -5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \\ \text{(a)} & \text{(b)} \end{array}$$

$$\begin{array}{cc} \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} & \begin{bmatrix} -1 & -1 & -1 \\ -1 & -8 & -1 \\ -1 & -1 & -1 \end{bmatrix} \\ \text{(c)} & \text{(d)} \end{array}$$

These four modes are sent via CSR instruction [Further explanation here] to the ISP block. The modes are as follows: (a) Pass-through; (b) Sharpen; (c) Gaussian Blur; (d) Edge-detect

The ISP block is the product of instantiating the same image convolution module three times (one for each color channel, pass alpha channel). This module has six 8-bit registers and accepts three 8-bit inputs per clock. This provides all the information to convolve using the 3x3 kernel. The matrix formed by the module:

$$\begin{bmatrix} t0 & t1 & \text{pix_top} \\ m0 & m1 & \text{pix_mid} \\ b0 & b1 & \text{pix_bot} \end{bmatrix}$$

t0, t1, m0, m1, b0, and b1 are the 8-bit storage registers and pix_top, pix_mid, pix_bot are the three input pixels accepted by the module.

The module runs the following algorithms to perform each convolution

- **Pass-Through:** Output is the center pixel, no reference to neighboring pixels:

$$\text{pass} = m1$$

- **Sharpen:** Enhances local contrast by subtracting neighboring pixels from a scaled center term:

$$\text{sharp} = 5m_1 - (t_1 + m_0 + \text{pix_mid} + b_1)$$

- **Gaussian Blur:** Performs low-pass filtering using a separable 3×3 Gaussian kernel:

$$\text{gauss} = \frac{1}{16} (t_0 + 2t_1 + \text{pix_top} + 2m_0 + 4m_1 + 2\text{pix_mid} + b_0 + 2b_1 + \text{pix_bot})$$

- **Edge Detection:** Accentuates large differences in intensity between neighboring pixels:

$$\text{edge} = 8m_1 - (t_0 + t_1 + \text{pix_top} + m_0 + \text{pix_mid} + b_0 + b_1 + \text{pix_bot})$$

One method to properly run this module is to pad the image pixel data with zeros on all the edges before feeding the image in. The transformation should be as follows.

$$\begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1N} \\ x_{21} & x_{22} & \cdots & x_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ x_{M1} & x_{M2} & \cdots & x_{MN} \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 & 0 \\ 0 & x_{11} & x_{12} & \cdots & x_{1N} & 0 \\ 0 & x_{21} & x_{22} & \cdots & x_{2N} & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & x_{M1} & x_{M2} & \cdots & x_{MN} & 0 \\ 0 & 0 & 0 & \cdots & 0 & 0 \end{bmatrix}$$

Where x_{mn} is the pixel data for that position in the image, M is the number of rows of pixel data, and N is the number of columns of pixel data.

Following this transformation, the pixel data is flattened and color channels are parsed in little-endian format. Flattened pixel data must be fed into the block from the top left to the bottom right of the data matrix. This implies a need to know the width and height of the image in order to capture the correct pixel on the clock. Pixel data can then be read from r16-r18 storing the data for each of the three rows being processed.

If the module receives a shift signal, it will shift the values for the center column registers into the left column registers and the inputs (right column) into the center column registers.

$$\begin{bmatrix} N/A & N/A & N/A \\ N/A & N/A & N/A \\ N/A & N/A & N/A \end{bmatrix} \rightarrow \begin{bmatrix} N/A & N/A & 0 \\ N/A & N/A & 0 \\ N/A & N/A & 0 \end{bmatrix}$$

$$\begin{bmatrix} N/A & 0 & 0 \\ N/A & 0 & x_{11} \\ N/A & 0 & x_{21} \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 0 & 0 \\ 0 & \boxed{x_{11}} & x_{12} \\ 0 & x_{21} & x_{22} \end{bmatrix}$$

The values N/A (not applicable) are listed since the processing window for those values should not be read as valid until the window is full. Once the window is full, the convolution run on the

center pixel, x_{11} , is valid and the shift signal should be zero. The output buffer accepts these valid outputs on the next clock edge when no shift signal is sent, after which the shift signal can again be set to 1.

At the end of each row, the reset signal must be set low to avoid erroneous overlap at the edges of the image.